Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

ПАРАДИГМЫ И КОНСТРУКЦИИ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

Конспект лекций по дисциплине

Москва – 2026

ОГЛАВЛЕНИЕ

1 ВВЕДЕНИЕ В ПАРАДИГМЫ И КОНСТРУКЦИИ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ 3

1.1 Сколько языков программирования существует и почему? 3
1.2 Классификации языков программирования 4
1.3 Что такое парадигма программирования? 4
1.4 Два больших подхода: императивный и декларативный 6
1.5 Императивный подход: процедурная и объектно-ориентированная парадигмы 7
1.5.1 Процедурное программирование 7
1.5.2 Объектно-ориентированное программирование 7
1.6 Декларативный подход: функциональная и логическая парадигмы 8
1.6.1 Функциональное программирование 8
1.6.2 Логическое программирование 9
1.7 Другие парадигмы 11
1.8 Выводы: почему же языков так много? 11

1 Введение в парадигмы и конструкции языков программирования

1.1 Сколько языков программирования существует и почему?

Точное количество языков программирования, существующих в мире, определить невозможно: итоговая цифра существенно зависит от критериев оценки и от самого определения понятия «язык программирования» [1]. Тем не менее порядок величины оценивается вполне определённо:

- по одним оценкам, общее число языков программирования колеблется в диапазоне от 1000 до 10 000, причём без учёта форков (диалектов одного и того же языка) эта величина сокращается примерно до 700 самостоятельных языков [1];
- по другим источникам, число языков программирования в мире достигает порядка 9000, и, в отличие от естественных языков, число которых сокращается, количество языков программирования продолжает расти [2].

Столь значительный разброс объясняется несколькими факторами:

- отсутствием единого критерия, отличающего самостоятельный язык от диалекта или библиотеки;
- включением или исключением из подсчёта эзотерических языков, созданных в учебных или развлекательных целях;
- постоянным появлением новых языков — как со стороны крупных технологических компаний, так и со стороны отдельных разработчиков [1].

Возникает закономерный вопрос: ***почему для решения задач программирования потребовалось такое количество различных инструментов?***

1.2 Классификации языков программирования

Прежде чем перейти к парадигмам, стоит вспомнить, что языки программирования вообще можно классифицировать по множеству независимых признаков:

- по уровню абстракции — низкоуровневые (ассемблер, машинный код) и высокоуровневые (Python, Java);
- по способу исполнения — компилируемые (C, Rust) и интерпретируемые (Python, JavaScript), хотя граница между ними всё более условна;
- по типизации — статически типизированные (C#, Rust, Haskell) и динамически типизированные (Python, Clojure, JavaScript);
- по специализации — языки общего назначения (C#, Python) и предметно-ориентированные языки, DSL (SQL, регулярные выражения, HTML);
- по форме представления — текстовые и визуальные (Scratch, LabVIEW).

Все эти классификации важны, но ни одна из них не отвечает на главный вопрос: как программист мыслит, когда пишет код на этом языке? Именно на этот вопрос отвечает классификация по парадигмам программирования, и именно она будет для нас основной на протяжении всего курса.

1.3 Что такое парадигма программирования?

Термин «парадигма программирования» был впервые применён в 1978 году Робертом Флойдом, который адаптировал его у философа науки

Томаса Куна (в книге «Структура научных революций» – https://ru.wikipedia.org/wiki/Структура_научных_революций). Но здесь есть важная тонкость: в отличие от научных парадигм по Куну, которые сменяют друг друга (после победы гелиоцентрической системы уже никто не пользуется геоцентрической), парадигмы программирования не являются взаимоисключающими. Флойд считал, что мастерство программиста растёт вместе с расширением его «репертуара» парадигм, а не заменой одной на другую.

Единого академического определения парадигмы программирования не существует — разные исследователи описывают её по-своему: как «семейство нотаций, разделяющих общий способ реализации программ» (Диомидис Спинеллис), как «стиль программирования, описывающий намерения программиста» (Дэниел Бобров), как «способ размышления о компьютерных системах» (Памела Зейв). Детальная информация приведена в википедии – https://ru.wikipedia.org/wiki/Парадигма_программирования

Мне более всего нравится определение Т. Бадда: **парадигма программирования – это «способ концептуализации того, что значит „производить вычисления“, и как задачи, подлежащие решению на компьютере, должны быть структурированы и организованы»**.

Важно, что современные языки программирования почти всегда мультипарадигмальны. Python позволяет писать и в объектно-ориентированном, и в функциональном стиле; C# — и в императивном, и (через LINQ) в декларативном. Поэтому вопрос «на каком языке лучше писать функционально?» часто менее важен, чем вопрос «какие конструкции языка позволяют мне удобно писать в этом стиле?» — и именно конструкциям языков будет посвящена практическая часть курса.

Основная задача конструкций языков программирования – наиболее удобным и компактным способом «поддержать» идеи, реализуемые парадигмой.

1.4 Два больших подхода: императивный и декларативный

Всё многообразие парадигм принято делить на два фундаментальных подхода, которые отвечают на разные вопросы.

Императивный подход отвечает на вопрос «как решить задачу?». Программист явно описывает последовательность шагов (алгоритм), которые должен выполнить компьютер, управляя при этом состоянием — набором переменных, которые меняются в процессе выполнения программы.

Декларативный подход отвечает на вопрос «что является результатом?». Программист описывает свойства желаемого результата, а как именно его получить — решает интерпретатор, компилятор или машина вывода.

Классическая иллюстрация разницы — задача «удвоить каждый элемент массива». Императивно на C мы напишем цикл:

```
// Императивный стиль: явно описываем ПОСЛЕДОВАТЕЛЬНОСТЬ ШАГОВ
void double_array(int *arr, int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * 2;   // изменяем состояние массива
    }
}
```

А декларативно, на языке SQL, мы вообще не напишем никакого цикла — мы просто описываем, какой результат хотим получить:

```
-- Декларативный стиль: описываем ЧТО хотим получить, а не КАК
SELECT value * 2 AS doubled_value FROM numbers;
```

Внутри каждого из этих двух подходов исторически сложились свои устойчивые парадигмы.

1.5 Императивный подход: процедурная и объектно-ориентированная парадигмы

1.5.1 Процедурное программирование

Исторически первая и самая простая парадигма: программа — это последовательность инструкций, объединённых в процедуры (подпрограммы), которые можно вызывать повторно. Данными и функциями, которые их обрабатывают, управляют раздельно.

Характерные языки: C, Pascal, Fortran, отчасти Go.

```
// Пример на C: типичная процедурная конструкция
// данные (struct) и функция существуют отдельно друг от друга
typedef struct { double a, b, c; } Coeffs;

double discriminant(Coeffs coef) {
    return coef.b * coef.b - 4 * coef.a * coef.c;
}
```

1.5.2 Объектно-ориентированное программирование

Развитие процедурной парадигмы, в котором данные и обрабатывающие их функции (методы) объединяются в единую сущность — объект. Ключевые понятия — инкапсуляция, наследование, полиморфизм.

Характерные языки: Java, C#, C++, Python (в объектном режиме), Ruby.

```
// Пример на C#: характерная ООП-конструкция
// инкапсуляция состояния и поведения в классе, наследование
abstract class GeometricFigure
{
```

```
    public abstract double Area();
}

class Circle : GeometricFigure
{
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
}
```

1.6 Декларативный подход: функциональная и логическая парадигмы

1.6.1 Функциональное программирование

В основе функциональной парадигмы лежит представление программы как вычисления математических функций без изменения состояния и без побочных эффектов. Функции здесь — «граждане первого класса»: их можно передавать как аргументы, возвращать из других функций, хранить в переменных. Что интересно, сами функциональные языки внутри себя делятся на несколько подвидов:

1. Однопарадигмальные. Используют только приемы функционального программирования. Примеры – Haskell, Erlang.
2. Объектно-функциональные. Наряду с ФП позволяют использовать ООП. Примеры – F#, OCaml, Scala.
3. Функционально-логические. Сочетают концепции функционального и логического программирования. Пример – Mercury
4. Мультипарадигмальные с развитым метапрограммированием. Используя метапрограммирование (поддержку макросов) как ядро языка существует возможность реализовать разные парадигмы, в том числе функциональную. Пример – LISP. Как правило такие языки обладают свойством самоотображаемости (гомоиконичности).

Свойство самоотображаемости (гомоиконичности) означает, что программа на таком языке представлена в виде структуры данных этого же языка — то есть код можно обрабатывать как данные, а данные превращать в исполняемый код.

Характерная конструкция функционального стиля — работа со списком без единого явного цикла, через рекурсию и функции высшего порядка:

```
// F#: типичная функциональная конструкция —
// рекурсивная обработка списка без изменяемых переменных
let rec sumOfSquares list =
    match list with
    | []      -> 0
    | x :: xs -> x * x + sumOfSquares xs
```

А вот как знаменитая быстрая сортировка выглядит на «чистом» функциональном языке Haskell — почти как математическое определение:

```
quickSort [] = []
quickSort (x:xs) = (quickSort lesser) ++ [x] ++ (quickSort greater)
    where lesser = filter (< p) xs
          greater = filter (>= p) xs
```

Характерные языки функциональной парадигмы: Haskell, Erlang, F#, OCaml, Scala, Clojure, Lisp, Scheme.

1.6.2 Логическое программирование

Ещё один декларативный подход, в котором программа — это не последовательность действий и не набор функций, а база знаний: множество фактов и правил вывода. Вместо того чтобы описывать алгоритм, программист формулирует утверждения, а задача решается путём автоматического логического вывода.

База знаний задается в виде предикатов. Правила вывода определяют связь между предикатами. Необходимо указать цель поиска и машина вывода пытается перебрать все комбинации предикатов, чтобы доказать

цель. Задача машины вывода – найти комбинацию исходных данных при которых цель выполняется или доказать что их нет.

Характерный язык: Prolog. Программа на нём выглядит совершенно не похоже ни на императивный, ни на функциональный код:

```
% Prolog: типичная конструкция логической парадигмы —
% факты и правила вместо алгоритма
родитель(анна, борис).
родитель(борис, виктор).

% правило: дедушка — это родитель родителя
дедушка(X, Y) :- родитель(X, Z), родитель(Z, Y).

% цель: ?- дедушка(анна, виктор).  →  true
```

Внутри логического программирования различают два принципиально разных механизма вывода:

1. Прямой логический вывод (от данных к цели, forward chaining): Программа записывается в виде продукционных правил «ЕСЛИ условие ТО действие». Порядок вызова правил определяется динамически машиной вывода. Цель динамически выводится в результате выполнения правил. Традиционно применяется в экспертных системах (язык CLIPS) и в современных системах программирования на основе правил (Business Rules Management Systems), например Drools.
2. Обратный логический вывод (от цели к данным, backward chaining) — механизм, реализованный в языке Prolog.

Связь логической и функциональной парадигм не ограничивается формальным сходством: между ними существует строгое математическое соответствие — изоморфизм Карри — Ховарда.

1.7 Другие парадигмы

Помимо четырёх рассмотренных «базовых» парадигм, существует ряд других — событийно-ориентированное, реактивное, предметно-ориентированное, автоматное программирование.

1.8 Выводы: почему же языков так много?

Теперь, вооружившись классификацией парадигм, мы можем ответить на вопрос, с которого начали. Языков программирования много именно потому, что разные парадигмы требуют разных языковых конструкций, а универсальный язык, одинаково удобный для всех парадигм сразу, создать крайне сложно.

Чтобы удобно писать в объектно-ориентированном стиле, языку нужны классы, наследование, инкапсуляция. Чтобы удобно писать функционально — неизменяемые данные, функции высшего порядка, сопоставление с образцом, алгебраические типы. Чтобы удобно писать логически — унификация, бэктрекинг, механизм фактов и правил. Ни один язык не может одинаково хорошо поддерживать всё это без компромиссов — а компромиссы разные разработчики видят по-разному. Отсюда и рождаются тысячи языков: каждый из них — это чья-то попытка найти свой баланс между парадигмами, удобством, производительностью и предметной областью.

Наш курс построен именно на этой идее. Мы будем разбирать конкретные языковые конструкции — алгебраические типы, сопоставление с образцом, функции высшего порядка, факты и правила вывода — и смотреть, как одни и те же задачи решаются на разных языках в разных парадигмах программирования. Такое сравнение — лучший способ по-настоящему понять, чем парадигмы отличаются друг от друга не на уровне определений, а на уровне кода.

В следующем разделе мы разберём практический пример: решение квадратного уравнения на языках C#, F#, Python и Rust в разных парадигмах программирования, чтобы увидеть, как одна и та же задача выглядит совершенно по-разному в зависимости от выбранной парадигмы.

Список источников раздела

1. Online Historical Encyclopaedia of Programming Languages (HOPL). — URL: https://hopl.info/
2. Зачем нам так много языков программирования? — URL: https://habr.com/ru/companies/mws/articles/806521/
3. Парадигма программирования // Википедия. — URL: https://ru.wikipedia.org/wiki/Парадигма_программирования